1 Kernel Density Estimation for Foreground Detection [25 Marks]



Figure 1: Foreground Detection Using KDE

1.1 Image Pre-Processing [2 Marks]

You are provided with two images:

- A **background image** (without any foreground object)
- A **test image** (with a foreground object present)

Your tasks are as follows:

1. **Image Alignment and Resizing:** Convert both images to the same dimensions, balancing computational efficiency and image quality.
2. **Feature Extraction:** Extract a relevant feature set (e.g., RGB channels, grayscale intensity, or color histograms) to be used as input for the KDE model.

Provide a brief justification for your chosen preprocessing approach and feature representation.

1.2 Custom KDE Class Implementation [15 Marks]

You are required to implement a custom **Kernel Density Estimation (KDE)** model from scratch, using only the `numpy` library (no external statistical or machine learning libraries are permitted).

Class Methods

1. ***__init__***
   Initialize the class with the following parameters:

   - `kernel`: a string specifying the kernel type (e.g., "gaussian", "triangular", "uniform").
   - `bandwidth`: a positive float controlling the smoothness of the density estimate.
   - `data`: the training data (e.g., background image features).

2. ***fit(data)***
   Implement the fitting method to store or preprocess the background data. Include a *smart sampling strategy* (e.g., random subset selection or grid-based sampling) to reduce computational cost while maintaining the representational quality of the model.

3. ***predict(samples)***
   Implement a prediction method that accepts a set of sample points (e.g., pixels from the test image) and returns their estimated probability densities under the fitted model. The prediction should use the chosen kernel function and bandwidth parameter to compute:

$$\hat{f}(x) = \frac{1}{nh^d} \sum_{i=1}^{n} K\left(\frac{x - x_i}{h}\right)$$

   where n is the number of training samples and d is the feature dimensionality.

1.3 Foreground Detection [8 Marks]

Use your KDE implementation to perform foreground detection as follows:

1. Fit the KDE model on the **background image features**.
2. Predict the density values for each pixel (or feature vector) in the **test image**.
3. Classify pixels as foreground or background based on an appropriate probability density threshold.

Experiment with different values of **bandwidth** and **kernel type** to obtain the best segmentation results. Provide a brief explanation for why your chosen parameter combination achieves superior performance.

For each class, briefly describe the architecture, hyperparameters, and optimization strategy used. Report the validation performance of all models on identical train/validation splits.

2 Data-Driven Discovery of a Discrete-Time Recurrence [20 Marks]

2.1 Dataset and Problem Setup [3 Marks]

You are provided with multiple univariate sequences $\{x_k\}_{k=0}^{T}$ generated by an **unknown, time-invariant, autonomous, discrete-time mechanism**. The data are noisily observed but follow a fixed rule within each sequence. Your goals are to (i) build predictors of x_k from its history, (ii) *identify* a compact analytical recurrence consistent with the data (including its effective order), and (iii) analyze how well it fits to the data. You are suggested to use an RNN for this task. Dataset is at Dataset link.

Tasks

1. **Data splits:** Create chronological train/validation/test splits. Justify lengths.
2. **Supervised pairs:** Form $(\mathbf{h}_k \mapsto x_k)$ with history vectors of length p and explain how p is selected.
3. **Normalization:** Apply normalization based on the training set only; describe inverse transform.

Suggested Implementation Skeleton

- `data_prep.py`: load data, create time splits, and normalize.
- `model.py`: implement baseline predictors (e.g., Linear AR, MLP, or small RNN).
- `train.py`: train chosen models, tune p, and log results.
- `identify.py`: derive analytical recurrence F_θ consistent with learned mapping.
- `analyze.py`: plot residuals, and report test performance.

2.2 Sequence Prediction [6 Marks]

Train one or more predictors mapping a chosen history to the next value:

$$\hat{x}_k = g_\phi\big(x_{k-1}, x_{k-2}, \ldots, x_{k-p}\big),$$

and later use this to recover a *recurrent* analytical expression. (Hint : *recurrent*)